JavaScript is disabled on your browser.

compact1, compact2, compact3

java.text

Interface CharacterIterator

All Superinterfaces:

Cloneable

All Known Subinterfaces:

AttributedCharacterIterator

All Known Implementing Classes:

Segment, StringCharacterIterator

```
public interface CharacterIterator
extends Cloneable
```

This interface defines a protocol for bidirectional iteration over text. The iterator iterates over a bounded sequence of characters. Characters are indexed with values beginning with the value returned by getBeginIndex() and continuing through the value returned by getEndIndex()-1.

Iterators maintain a current character index, whose valid range is from getBeginIndex() to getEndIndex(); the value getEndIndex() is included to allow handling of zero-length text ranges and for historical reasons. The current index can be retrieved by calling getIndex() and set directly by calling setIndex(), first(), and last().

The methods previous() and next() are used for iteration. They return DONE if they would move outside the range from getBeginIndex() to getEndIndex() -1, signaling that the iterator has reached the end of the sequence. DONE is also returned by other methods to indicate that the current index is outside this range.

Examples:

Traverse the text from start to finish

```
public void traverseForward(CharacterIterator iter) {
    for(char c = iter.first(); c != CharacterIterator.DONE; c = iter.next()) {
        processChar(c);
    }
}
```

Traverse the text backwards, from end to start

```
public void traverseBackward(CharacterIterator iter) {
    for(char c = iter.last(); c != CharacterIterator.DONE; c = iter.previous()) {
        processChar(c);
    }
}
```

Traverse both forward and backward from a given position in the text. Calls to notBoundary() in this example represents some additional stopping criteria.

```
public void traverseOut(CharacterIterator iter, int pos) {
    for (char c = iter.setIndex(pos);
             c != CharacterIterator.DONE && notBoundary(c);
             c = iter.next()) {
    }
    int end = iter.getIndex();
    for (char c = iter.setIndex(pos);
            c != CharacterIterator.DONE && notBoundary(c);
            c = iter.previous()) {
    }
    int start = iter.getIndex();
    processSection(start, end);
}
```

See Also:

StringCharacterIterator, AttributedCharacterIterator

Field Summary

Fields

Modifier and Type	Field	Description
static char	DONE	Constant that is returned when the iterator has reached either the end or the beginning of the text.

Method Summary

Abstract Methods

Modifier and Type	Method	Description
Object	clone()	Create a copy of this iterator
char	current()	Gets the character at the current position (as returned by getIndex()).
char	first()	Sets the position to getBeginIndex() and returns the character at that position.
int	getBeginIndex()	Returns the start index of the text.
int	getEndIndex()	Returns the end index of the text.
int	getIndex()	Returns the current index.
char	last()	Sets the position to getEndIndex()-1 (getEndIndex() if the text is empty) and returns the character at that position.
char	next()	Increments the iterator's index by one and returns the character at the new index.
char	previous()	Decrements the iterator's index by one and returns the character at the new index.
char	setIndex(int position)	Sets the position to the specified position in the text and returns that character.

Field Detail

DONE

```
static final char DONE
```

Constant that is returned when the iterator has reached either the end or the beginning of the text. The value is '\\uFFFF', the "not a character" value which should not occur in any valid Unicode string.

See Also:

Constant Field Values

Method Detail

first

```
char first()
```

Sets the position to getBeginIndex() and returns the character at that position.

Returns:

the first character in the text, or DONE if the text is empty

See Also:

getBeginIndex()

last

```
char last()
```

Sets the position to getEndIndex()-1 (getEndIndex() if the text is empty) and returns the character at that position.

Returns:

the last character in the text, or DONE if the text is empty

See Also:

getEndIndex()

current

```
char current()
```

Gets the character at the current position (as returned by getIndex()).

Returns:

`the character at the current position or DONE if the current position is off the end of the text.`

See Also:

`getIndex()`

next

```
char next()
```

Increments the iterator's index by one and returns the character at the new index. If the resulting index is greater or equal to getEndIndex(), the current index is reset to getEndIndex() and a value of DONE is returned.

Returns:

`the character at the new position or DONE if the new position is off the end of the text range.`

previous

```
char previous()
```

Decrements the iterator's index by one and returns the character at the new index. If the current index is getBeginIndex(), the index remains at getBeginIndex() and a value of DONE is returned.

Returns:

`the character at the new position or DONE if the current position is equal to getBeginIndex().`

setIndex

```
char setIndex(int position)
```

Sets the position to the specified position in the text and returns that character.

Parameters:

`position - the position within the text. Valid values range from getBeginIndex() to getEndIndex(). An IllegalArgumentException is thrown if an invalid value is supplied.`

Returns:

`the character at the specified position or DONE if the specified position is equal to getEndIndex()`

getBeginIndex

```
int getBeginIndex()
```

Returns the start index of the text.

Returns:

`the index at which the text begins.`

getEndIndex

```
int getEndIndex()
```

Returns the end index of the text. This index is the index of the first character following the end of the text.

Returns:

`the index after the last character in the text`

getIndex

```
int getIndex()
```

Returns the current index.

Returns:

`the current index.`

clone

```
Object clone()
```

Create a copy of this iterator

Returns:

A copy of this

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.